МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Одеський національний університет імені І. І. Мечникова
Факультет математики, фізики та інформаційних технологій
Кафедра оптимального керування та економічної кібернетики

ЗВІТ

з лабораторної роботи № 1
з дисципліни «Системи комп’ютерного зору (частина 1)»
на тему: «Середовище, подання зображень, колірні простори»

Виконала: студентка 4 курсу
спеціальності 113 «Прикладна математика»
Граб Ульяна Олександрівна

Перевірив: канд. техн. наук, доц.
Мороз В.

Одеса – 2026

1. Мета роботи

Налаштувати робоче середовище; дослідити особливості цифрового подання зображень (типи даних, порядок каналів, втрати при стисненні); кількісно порівняти стабільність порогового виділення об’єкта в колірних просторах RGB, HSV і Lab за зміни освітлення; дослідити аліасинг при зменшенні, методи інтерполяції при збільшенні та вплив гамма-кодування.

2. Вихідні дані

- Камера: iPhone 14 Pro, основна камера (6,86 мм, еквівалент 24 мм, f/1.78). Знімки передано на комп’ютер у роздільності 2048×1536 (3 Мп), метадані EXIF збереглися.
- Програмне забезпечення: Python 3.14, OpenCV 5.0.0 (opencv-contrib-python), NumPy 2.5.4, scikit-image 0.26.0, Matplotlib 3.11.2, Pillow 12.3.0. Версії зафіксовано в requirements.txt, випадковий стан – np.random.seed(42).
- photo.jpg – пейзаж із небом і лісом (частини A і C); texture.jpg – вишивка хрестиком на канві, дрібна регулярна сітка (крок 12).
- object_light1.jpg / object_light2.jpg – помаранчева баночка меду на синьому кріслі: кадр 1 – природне світло з вікна, кадр 2 – тепле світло лампи. Між кадрами телефон трохи зсунувся, тому еталонну маску зроблено для кожного кадру окремо.
- Власний набір – 20 кадрів однієї текстурованої сцени (сад біля будинку: будинки, паркан, дерева). 10 кадрів повороту камери навколо вертикальної осі з однієї точки нарізано з відео IMG_1332 (1080×1920, 60 кадр/с, 29.07.2026): з околу ±10 кадрів навколо рівномірно розташованих моментів брався найрізкіший (максимум дисперсії лапласіана). Ще 10 – фотографії тієї самої сцени з різних ракурсів за різного освітлення (день, хмарно, захід сонця, контрове сонце), дві з них – із дворазовим наближенням. Кадри відео EXIF не мають, тому для них збережено метадані самого відео (камера, час початку запису) і обчислено час кожного кадру як початок запису плюс його положення у відео; витримка й ISO у відеофайлі не зберігаються. Параметри всіх 20 кадрів – у таблиці 2 і results/a3_exif_dataset.csv. Фото зроблено у квітні–вересні, тому листя на деревах різне.

Таблиця 1 – Параметри зйомки за даними EXIF

Файл	Фокусна відстань	Діафрагма	Витримка	ISO	Дата
photo.jpg	6,86 мм (24 мм)	f/1.78	1/864 с	80	11.10.2026
texture.jpg	6,86 мм (24 мм)	f/1.78	1/50 с	400	01.09.2026
object_light1.jpg	6,86 мм (24 мм)	f/1.78	1/33 с	640	15.10.2026
object_light2.jpg	6,86 мм (24 мм)	f/1.78	1/50 с	320	15.10.2026

Таблиця 2 – Параметри зйомки кадрів власного набору

Файл	Екв. 35 мм	Витримка	ISO	Дата	Освітлення
scene_IMG_0160	24 мм	1/103 с	250	29.04.2026	захід сонця
scene_IMG_0169	48 мм	1/41667 с	50	30.04.2026	контрове сонце
scene_IMG_0252	24 мм	1/220 с	64	10.05.2026	вечір, хмари
scene_IMG_0663	24 мм	1/121 с	64	14.06.2026	хмарний вечір
scene_IMG_0664	24 мм	1/121 с	125	14.06.2026	хмарний вечір
scene_IMG_1328	32 мм	1/100 с	125	29.07.2026	день, хмарно
scene_IMG_1331	32 мм	1/100 с	125	29.07.2026	день, хмарно
scene_IMG_2217	24 мм	1/121 с	80	27.08.2026	захід, хмари
scene_IMG_2218	24 мм	1/221 с	64	27.08.2026	захід, проти сонця
scene_IMG_2559	48 мм	1/4184 с	64	16.09.2026	хмарно
rot_01 … rot_10	–	–	–	29.07.2026, 16:28:09,4 … 16:28:18,8	день, хмарно; кадри відео IMG_1332

3. Теоретичні відомості

PSNR = 10·lg(255² / MSE), де MSE – середній квадрат різниці значень пікселів двох зображень; для однакових зображень PSNR = ∞.

IoU = $|A \cap B| / |A \cup B|$ – відношення площі перетину отриманої та еталонної масок до площі їх об’єднання; accuracy – частка пікселів, у яких маска збігається з еталоном.

Гамма-корекція: out = 255·(in / 255)^γ. Значення пікселів у файлах зберігаються гамма-кодованими, фізична освітленість L ≈ (v / 255)^2,2, тому лінійні операції над кодами v не еквівалентні операціям над освітленістю.

При зменшенні зображення в k разів частоти, вищі за нову частоту Найквіста 0,5 / k циклів на піксель, мають бути попередньо подавлені фільтром нижніх частот, інакше вони відображаються як хибні низькочастотні візерунки (муар).

4. Хід роботи

Частина A. Зчитано photo.jpg, виведено властивості масиву, показано зображення без перетворення BGR→RGB і з ним. Яскравість підвищено на 50 трьома способами: img + 50 в uint8, cv2.add(img, 50) та через int16 з np.clip. Зображення збережено в PNG і в JPEG із якістю 5…100 з кроком 5; для кожного файлу обчислено розмір і PSNR відносно оригіналу (власна функція, звірена з cv2.PSNR). EXIF прочитано через PIL.ExifTags.

Частина B. Еталонні маски отримано методом GrabCut із прямокутником навколо баночки (5 ітерацій) і перевірено візуально. Межі порогування для кожного каналу підбиралися на кадрі 1 як 1-й і 99-й перцентилі значень пікселів об’єкта; ті самі межі застосовувалися до кадру 2 через cv2.inRange. Крім RGB і повного HSV, перевірено варіанти без каналу яскравості: HSV лише за H і S та Lab лише за a і b.

Частина C. Аліасинг досліджено на вишивці та на синтетичній зонній пластинці 1024×1024 з інтенсивністю 127,5 + 127,5·cos(πr²/N), у якої локальна частота f = r / N відома в кожній точці. Зменшення в 4 рази виконано чотирма способами: INTER_NEAREST, GaussianBlur (σ = 2) + INTER_NEAREST, INTER_AREA і два рівні піраміди зображень

(cv2.pyrDown). Мірою муару взято стандартне відхилення в зоні 0,15 < f < 0,48 (там правильний результат – рівномірно сірий), мірою втрати деталей – частку збереженого контрасту в зоні f < 0,075. Для інтерполяції фрагмент 512×512 зменшено в 8 разів (INTER_AREA) і збільшено назад чотирма методами; обчислено PSNR, SSIM і медіану часу з 20 запусків. Побудовано передавальні характеристики гамма-корекції для γ = 0,5; 1,0; 2,2, обчислено середні в кодах і в освітленості, застосовано баланс білого «сірий світ» до кадру під лампою.

Основні функції, використані в роботі, наведено в таблиці 3; повний код із коментарями – у додатку А.

Таблиця 3 – Основні функції бібліотек

Функція	**Що робить**	**Як використано**
cv2.imread	зчитує файл у масив NumPy (H, W, 3), канали B, G, R, тип uint8	усі зображення; маски – з IMREAD_GRAYSCALE
cv2.cvtColor	перетворює колірний простір	BGR → RGB для показу, BGR → GRAY, BGR → HSV, BGR → Lab
cv2.add	додавання з насиченням: результат обмежується 0…255	підвищення яскравості на 50
np.clip	обмежує значення масиву заданим діапазоном	0…255 після обчислення в int16
cv2.imwrite	зберігає зображення; формат – за розширенням	PNG і JPEG, IMWRITE_JPEG_QUALITY = 5…100
cv2.PSNR	бібліотечний PSNR	звірка власної функції psnr()
Image.getexif, get_ifd	читають метадані EXIF (Pillow)	витримка, ISO, фокусна відстань, дата
cv2.grabCut	відділяє об’єкт від фону за прямокутником (моделі кольорів + розріз графа)	еталонні маски, 5 ітерацій
np.percentile	значення, нижче якого лежить q % вибірки	межі порогів: 1/99 (а також 2/98, 5/95)
cv2.inRange	маска 255, якщо всі канали пікселя в межах [low, high]	порогування в RGB, HSV, Lab
cv2.resize	змінює розмір із заданою інтерполяцією	NEAREST, LINEAR, CUBIC, LANCZOS4, AREA
cv2.GaussianBlur	гауссове згладжування; розмір ядра обчислюється з σ	σ = 2 перед зменшенням у 4 рази
cv2.pyrDown	рівень піраміди: гаус 5×5 і кожен другий піксель (зменшення вдвічі)	двічі – зменшення в 4 рази
structural_similarity	індекс SSIM (scikit-image)	порівняння інтерполяцій, channel_axis = 2
cv2.LUT	замінює значення пікселів за таблицею з 256 елементів	гамма-корекція
cv2.Laplacian	друга похідна зображення; її дисперсія – міра різкості	вибір кадрів із відео

5. Результати

5.1. Подання зображення

Зображення зчитується як масив розміром (1536, 2048, 3) типу uint8 зі значеннями 0… 255, обсяг у пам’яті 9,0 МБ. Без перетворення BGR→RGB червоні й сині канали на рисунку переставлені. Наївне додавання img + 50 спричинило переповнення у 7,9 % пікселів, з них 3,36 % пікселів із яскравих стали темними (рис. 1). Результати cv2.add(img, 50) та int16 + np.clip повністю збіглися.

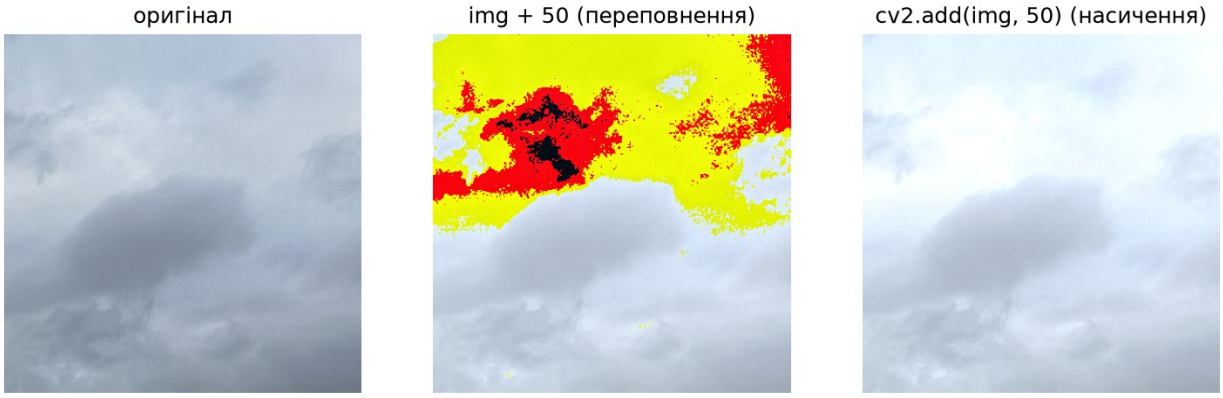


Рисунок 1 – Фрагмент неба: переповнення uint8 при img + 50 і коректне насичення cv2.add

Таблиця 4 – Втрати при стисненні (оригінал 9216 КБ без стиснення)

Формат	Якість	Розмір, КБ	Стиснення, разів	PSNR, дБ
PNG	–	4935,2	1,9	∞
JPEG	95	1145,6	8,0	47,78
JPEG	75	496,3	18,6	35,82
JPEG	50	337,4	27,3	32,90
JPEG	25	222,9	41,3	30,25
JPEG	10	122,5	75,2	26,81

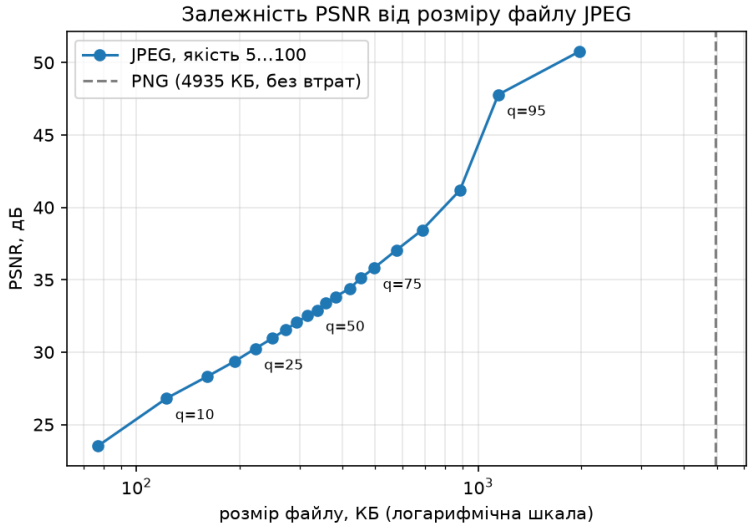


Рисунок 2 – Залежність PSNR від розміру файлу JPEG

5.2. Колірні простори



Рисунок 3 – Кадри з різним освітленням та еталонні маски

Таблиця 5 – Підібрані межі та метрики відносно еталонних масок

Простір	Межі (нижні … верхні)	IoU, кадр 1	accuracy, кадр 1	IoU, кадр 2	accuracy, кадр 2
RGB (B, G, R)	[0, 8, 19] … [142, 166, 200]	0,288	0,447	0,105	0,214
HSV	[8, 37, 19] … [24, 255, 200]	0,943	0,987	0,371	0,836
HSV (H, S)	H 8…24, S 37…255	0,955	0,989	0,846	0,960
Lab (a, b)	a 127…153, b 130…183	0,945	0,987	0,685	0,917

Щоб перевірити, чи не залежить висновок від вибору перцентилів, межі також підбиралися за перцентилями 2–98 і 5–95 (табл. 6).

Таблиця 6 – IoU на кадрі 2 при різних перцентилях для підбору меж

Простір	1–99	2–98	5–95
RGB	0,105	0,096	0,088
HSV	0,371	0,296	0,209
HSV (H, S)	0,846	0,829	0,760
Lab (a, b)	0,685	0,648	0,589

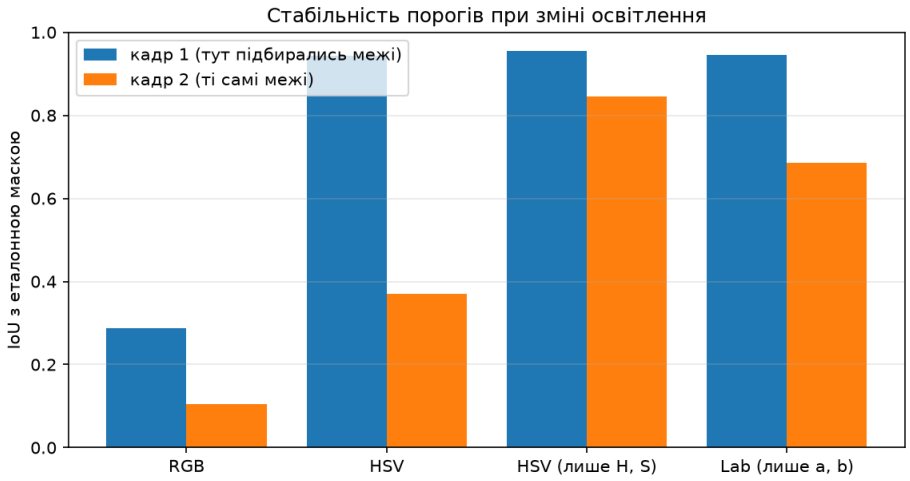


Рисунок 4 – IoU на кадрі, де підбиралися межі, і на кадрі з іншим освітленням

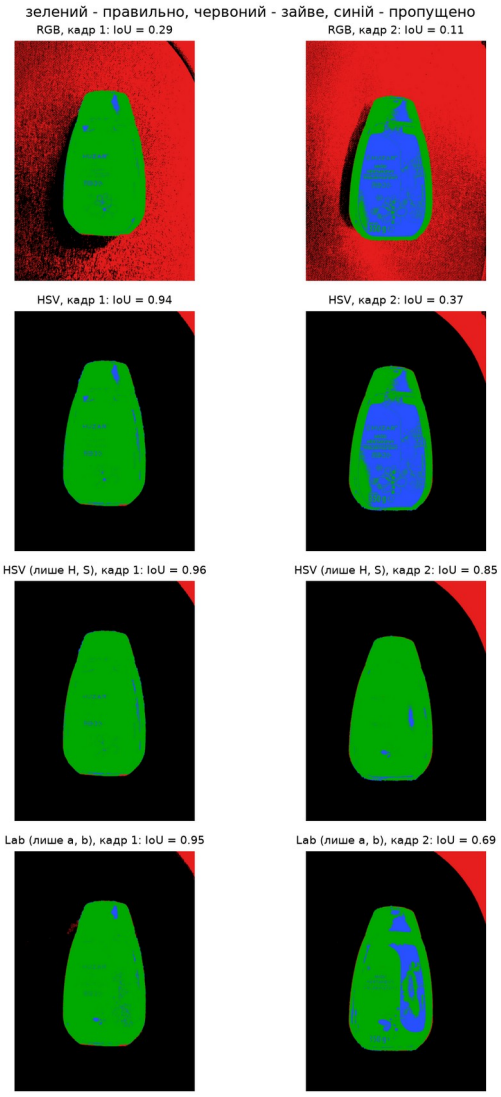


Рисунок 5 – Помилки сегментації: зелений – правильно, червоний – зайве, синій – пропущено

5.3. Аліасинг

Таблиця 7 – Зменшення зонної пластинки в 4 рази

Метод	Муар: std у зоні f > f_H, рівні	Збережений контраст низьких частот, %
INTER_NEAREST	90,1	100,3
GaussianBlur (σ = 2) + NEAREST	2,9	82,1
INTER_AREA	16,0	94,5
pyrDown × 2 (піраміда)	1,0	78,2

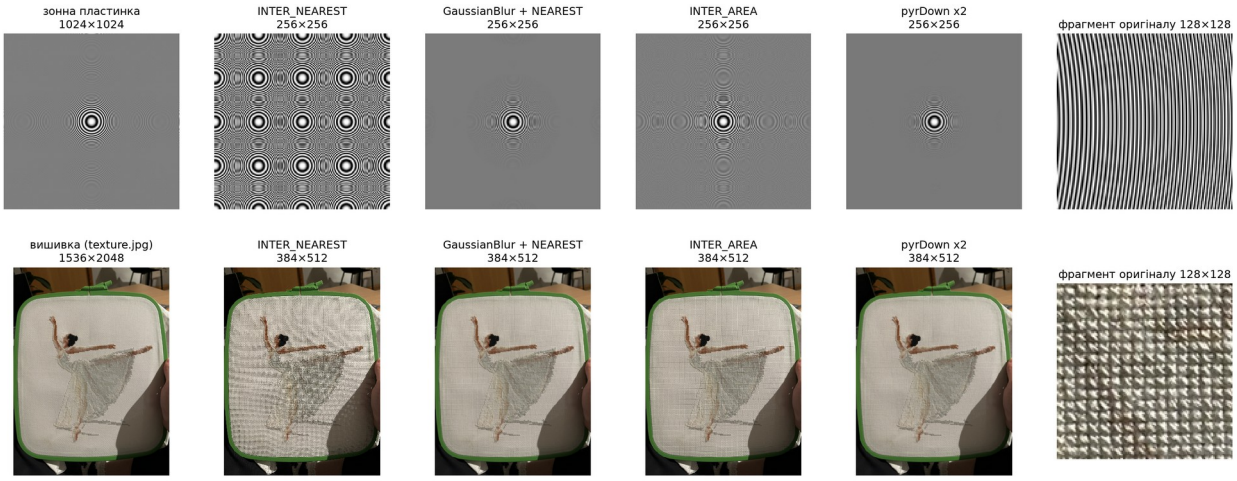


Рисунок 6 – Зменшення в 4 рази: зонна пластинка (угорі) і вишивка (унизу)

5.4. Інтерполяція

Для оцінки розкиду збільшення повторено на 5 фрагментах 512×512 уздовж горизонту (табл. 8: середнє ± стандартне відхилення по фрагментах; час – медіана з 20 запусків ± стандартне відхилення).

Таблиця 8 – Збільшення фрагментів 64×64 → 512×512 (5 фрагментів)

Метод	PSNR, дБ	SSIM	Час, мс
INTER_NEAREST	23,11 ± 0,60	0,450 ± 0,014	0,110 ± 0,021
INTER_LINEAR	23,27 ± 0,66	0,463 ± 0,016	0,092 ± 0,031
INTER_CUBIC	23,44 ± 0,69	0,475 ± 0,016	0,175 ± 0,014
INTER_LANCZOS4	23,45 ± 0,69	0,475 ± 0,017	2,151 ± 0,213



Рисунок 7 – Збільшення в 8 разів різними методами (один із фрагментів)

5.5. Гамма та баланс білого

Таблиця 9 – Усереднення двох пікселів у кодах і в освітленості (γ = 2,2)

v_1	v_2	Середнє кодів	Середнє освітленостей (у кодах)	Різниця
0	255	127,5	186,1	58,6
50	200	125,0	149,1	24,1
100	150	125,0	128,0	3,0
0	100	50,0	73,0	23,0

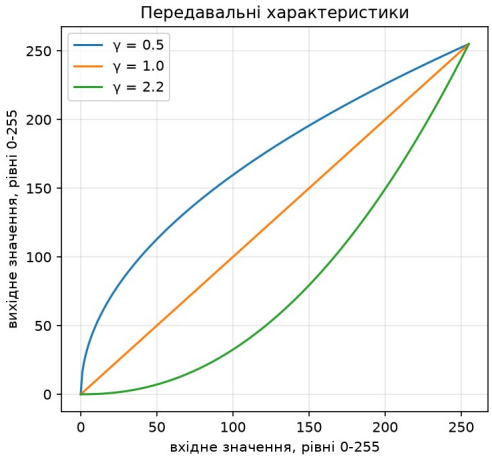


Рисунок 8 – Передавальні характеристики гамма-корекції

Баланс білого «сірий світ» вирівняв середні каналів кадру під лампою: до корекції B = 84,3, G = 90,5, R = 95,6 (R/B = 1,13), після – 89,6, 89,5, 89,6 (R/B = 1,00).

6. Аналіз

Переповнення. Тип uint8 рахує за модулем 256, тому значення 206…255 після додавання 50 «перескакують» у 0…49. Це зачіпає саме найсвітліші ділянки (небо), де 4,25 % значень каналів ≥ 206, і дає кольорові та темні плями. Для коректного результату потрібне насичення (cv2.add) або обчислення в ширшому типі.

Стиснення. PSNR спадає майже лінійно з логарифмом розміру файлу. Перехід від якості 95 до 75 зменшує файл у 2,3 раза ціною 12 дБ, але візуально різниці майже немає; при якості 25 і нижче з’являються блоки 8×8. PNG зберігає зображення без втрат, але стискає фото лише в 1,9 раза.

Колірні простори. RGB непридатний уже на кадрі 1 (IoU 0,288): об’єкт містить і темні пікселі меду, і світлі пікселі етикетки, тому межі по кожному каналу виходять широкими (0…200) і захоплюють темно-синє крісло (рис. 5, червоне). Кадр 2 знятий яскравіше (за EXIF ISO 320 проти 640), тому частина об’єкта виходить за верхні межі, і IoU падає до 0,105. Повний HSV на кадрі 1 майже ідеальний (0,943), але на кадрі 2 падає до 0,371: світліший мед і етикетка перевищують верхню межу V = 200 (рис. 5, синє). Без каналу V результат найстабільніший: 0,955 → 0,846, оскільки відтінок меду під обома освітленнями лишається в межах H = 8…24; залишкові помилки – відблиск лампи (низька насиченість) і край крісла. Висновок не залежить від вибору перцентилів: за 1–99, 2–98 і 5–95 порядок просторів однаковий, а HSV (H, S) лишається найкращим (0,846; 0,829; 0,760). Lab за a, b погіршився сильніше (0,945 → 0,685): тепле світло зсуває хроматичні координати, і світла етикетка виходить за межі. Accuracy тут оманлива: порожня маска на кадрі 2 дала б 0,773, а HSV з accuracy 0,836 має IoU лише 0,371.

Аліасинг. При INTER_NEAREST частоти вище нової межі Найквіста не зникають, а повністю зберігають амплітуду (std 90,1 ≈ 127,5/√2) і стають хибними кільцями. На вишивці сітка канви з періодом близько 6 пікселів після зменшення в 4 рази має період ~1,5 пікселя, тобто вище межі Найквіста, і перетворюється на муар. Гауссове згладжування σ = 2 прибирає муар (2,9), але втрачає 18 % контрасту корисних деталей. Піраміда (pyrDown – гаус 5×5 і проріджування вдвічі, двічі) подавлює муар найсильніше (1,0) ціною 22 % контрасту. INTER_AREA (усереднення блоків 4×4) – компроміс: 16,0 і 94,5 %, але на вишивці після нього ще видно сітку.

Інтерполяція. Деталі, втрачені при зменшенні у 8 разів, жоден метод не відновлює, тому різниця між методами в середньому лише 0,34 дБ – менша за розкид між фрагментами (±0,6… 0,7 дБ). Проте на кожному з 5 фрагментів порядок однаковий: NEAREST < LINEAR < CUBIC ≈ LANCZOS4, тобто перевага кубічної інтерполяції стабільна. INTER_CUBIC і INTER_LANCZOS4 практично однакові за якістю, але Lanczos повільніший приблизно в 12 разів; INTER_NEAREST дає видимі блоки, INTER_LINEAR – розмиття. На фрагментах із самим листям або сіткою канви різниця між методами зникала (≈ 0,1 дБ): там після зменшення лишається лише дрібна текстура.

Гамма та баланс білого. Середнє кодів 0 і 255 дорівнює 127,5, а фізично середня освітленість відповідає коду 186 – різниця 58,6 рівня; для близьких значень (100 і 150) вона лише 3 рівні. Тому розмиття чи зменшення в гамма-кодованому просторі затемнює контрастні межі. «Сірий світ» прибрав теплий відтінок (R/B 1,13 → 1,00), але візуально ефект невеликий, бо телефон уже виконав автоматичний баланс білого; крім того, припущення про сірий середній колір порушується, коли кадр займає синє крісло.

7. Висновки

1. Арифметика над uint8 без приведення типу зіпсувала 7,9 % пікселів фото, переважно на небі; cv2.add і обчислення в int16 з np.clip дають однаковий коректний результат.
2. JPEG з якістю 75 зменшує файл у 18,6 раза при PSNR 35,8 дБ, з якістю 10 – у 75 разів при 26,8 дБ і видимих блоках 8×8; PNG без втрат стискає лише в 1,9 раза.
3. Найстабільнішим до зміни освітлення виявився поріг за H і S: IoU 0,955 → 0,846; повний HSV через канал V погіршився з 0,943 до 0,371, а RGB непридатний навіть на кадрі підбору (0,288 → 0,105).
4. Зменшення без згладжування дає муар (std 90,1 на зонній пластинці замість 0); гауссове згладжування σ = 2 зменшує його до 2,9, піраміда pyrDown – до 1,0 ціною 18 і 22 % контрасту відповідно; INTER_AREA – компроміс (16,0 і 94,5 %).
5. При збільшенні в 8 разів методи відрізняються в середньому на 0,34 дБ (розкид між фрагментами ±0,6 дБ), але порядок стабільний на всіх фрагментах; INTER_CUBIC – найкращий баланс якості й швидкості (Lanczos у 12 разів повільніший без виграшу). Усереднення гамма-кодованих значень занижує яскравість до 58,6 рівня.

8. Декларація про використання генеративного ШІ

Під час виконання роботи як допоміжний інструмент використовувався генеративний ШІ (Claude, Anthropic): для консультацій щодо методики експериментів, допомоги в написанні й налагодженні коду Python та в оформленні звіту. Усі результати отримано запуском коду на власних даних; відповідальність за коректність коду й висновків несе автор звіту.

Додаток А. Програмний код

Код запускається з папки lab01/src: python part_a.py, python part_b.py, python part_c.py. Результати зберігаються в results/ (CSV) і figures/ (PNG).

А.1. Спільні функції – utils.py

```
# спільне для всіх частин

import csv
import os

import numpy as np

# папка lab01
LAB = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA = os.path.join(LAB, "data")
RESULTS = os.path.join(LAB, "results")
FIG = os.path.join(LAB, "figures")
TMP = os.path.join(RESULTS, "tmp")  # тимчасові jpg/png

os.makedirs(RESULTS, exist_ok=True)
os.makedirs(FIG, exist_ok=True)
os.makedirs(TMP, exist_ok=True)



def psnr(original, test):
    # 10 * log10(255^2 / MSE)
    mse = np.mean((original.astype(np.float64) - test.astype(np.float64)) ** 2)
    if mse == 0:
        return float("inf")
    return 10 * np.log10(255 ** 2 / mse)



def iou(pred, gt):
    pred = pred > 0
    gt = gt > 0
    intersection = np.logical_and(pred, gt).sum()
    union = np.logical_or(pred, gt).sum()
    return intersection / union



def accuracy(pred, gt):
    return np.mean((pred > 0) == (gt > 0))



def save_csv(name, header, rows):
    with open(os.path.join(RESULTS, name), "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(rows)
```

А.2. Частина А – part_a.py

```
# ЛР1, частина А, кроки 1-5

import csv
import os
from datetime import datetime, timedelta

import cv2
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from PIL import ExifTags, Image

from utils import DATA, FIG, TMP, psnr, save_csv

np.random.seed(42)

img = cv2.imread(os.path.join(DATA, "photo.jpg"))
```

```
# Крок 1. Властивості масиву
print("Крок 1")
print("shape:", img.shape)
print("dtype:", img.dtype)
print("min:", img.min(), "max:", img.max())
print("пам'ять, МБ:", round(img.nbytes / 1024 / 1024, 2))


save_csv("a1_properties.csv", ["параметр", "значення"], [
    ["shape (H, W, C)", str(img.shape)],
    ["dtype", str(img.dtype)],
    ["min", img.min()],
    ["max", img.max()],
    ["обсяг у пам'яті, МБ", round(img.nbytes / 1024 / 1024, 2)],
])



# Крок 2. BGR і RGB
# OpenCV - BGR, matplotlib - RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)


fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].imshow(img)
ax[0].set_title("plt.imshow(img) без перетворення\n(канали R і B переставлені)")
ax[1].imshow(img_rgb)
ax[1].set_title("після cv2.cvtColor(img, cv2.COLOR_BGR2RGB)")
ax[0].axis("off")
ax[1].axis("off")
plt.savefig(os.path.join(FIG, "a1_bgr_vs_rgb.png"), dpi=150, bbox_inches="tight")
plt.close()



# Крок 3. Переповнення uint8
wrong = img + 50  # 250 + 50 = 44
right = cv2.add(img, 50)  # 250 + 50 = 255
right2 = np.clip(img.astype(np.int16) + 50, 0, 255).astype(np.uint8)
print("\nКрок 3")
print("cv2.add і int16 + clip дають однаковий результат:", np.array_equal(right, right2))


overflow = (wrong < img).any(axis=2)  # хоч один канал переповнився
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_wrong = cv2.cvtColor(wrong, cv2.COLOR_BGR2GRAY)
bright_to_dark = (gray > 200) & (gray_wrong < 100)


print("пікселів з переповненням:", round(overflow.mean() * 100, 2), "%")
print("яскраві стали темними:", round(bright_to_dark.mean() * 100, 2), "%")
save_csv("a1_overflow.csv", ["метод", "пікселів з переповненням, %", "яскраві→темні, %"], [
    ["img + 50", round(overflow.mean() * 100, 2), round(bright_to_dark.mean() * 100, 2)],
    ["cv2.add(img, 50)", 0, 0],
])


# шматок неба
y1, y2, x1, x2 = 0, 384, 1388, 1772
fig, ax = plt.subplots(1, 3, figsize=(12, 4.5))
titles = ["оригінал", "img + 50 (переповнення)", "cv2.add(img, 50) (насичення)"]
for i, im in enumerate([img, wrong, right]):
    ax[i].imshow(cv2.cvtColor(im[y1:y2, x1:x2], cv2.COLOR_BGR2RGB))
    ax[i].set_title(titles[i])
    ax[i].axis("off")
plt.savefig(os.path.join(FIG, "a1_overflow.png"), dpi=150, bbox_inches="tight")
plt.close()



# Крок 4. Втрати при стисненні
print("\nКрок 4")
raw_size = img.nbytes / 1024  # КБ


cv2.imwrite(os.path.join(TMP, "photo.png"), img)
png_size = os.path.getsize(os.path.join(TMP, "photo.png")) / 1024
png_back = cv2.imread(os.path.join(TMP, "photo.png"))
print("PNG:", round(png_size), "КБ, без втрат:", np.array_equal(png_back, img))


table = [["PNG", "-", round(png_size, 1), round(raw_size / png_size, 1), "inf"]]
sizes = []
psnrs = []
qualities = list(range(5, 101, 5))  # для графіка більше точок
for q in qualities:
```

```
    path = os.path.join(TMP, f"photo_q{q}.jpg")
    cv2.imwrite(path, img, [cv2.IMWRITE_JPEG_QUALITY, q])
    size = os.path.getsize(path) / 1024
    back = cv2.imread(path)
    p = psnr(img, back)
    sizes.append(size)
    psnrs.append(p)
    if q in [95, 75, 50, 25, 10]:
        print(f"JPEG q={q}: {size:.1f} КБ, PSNR = {p:.2f} дБ (cv2.PSNR = {cv2.PSNR(img,
back):.2f})")
        table.append(["JPEG", q, round(size, 1), round(raw_size / size, 1), round(p, 2)])

save_csv("a2_compression.csv", ["формат", "якість", "розмір, КБ", "стиснення, разів", "PSNR, дБ"],
table)


plt.figure(figsize=(7, 4.5))
plt.plot(sizes, psnrs, "o-", label="JPEG, якість 5...100")
for q, s, p in zip(qualities, sizes, psnrs):
    if q in [95, 75, 50, 25, 10]:
        plt.annotate(f"q={q}", (s, p), xytext=(5, -12), textcoords="offset points", fontsize=8)
plt.axvline(png_size, color="gray", linestyle="--", label=f"PNG ({png_size:.0f} КБ, без втрат)")
plt.xscale("log")
plt.xlabel("розмір файлу, КБ (логарифмічна шкала)")
plt.ylabel("PSNR, дБ")
plt.title("Залежність PSNR від розміру файлу JPEG")
plt.grid(True, which="both", alpha=0.3)
plt.legend()
plt.savefig(os.path.join(FIG, "a2_psnr_vs_size.png"), dpi=150, bbox_inches="tight")
plt.close()

# артефакти
y1, y2, x1, x2 = 600, 856, 900, 1156
fig, ax = plt.subplots(1, 3, figsize=(12, 4.3))
for i, q in enumerate([None, 50, 10]):
    if q is None:
        part = img[y1:y2, x1:x2]
        ax[i].set_title("оригінал")
    else:
        part = cv2.imread(os.path.join(TMP, f"photo_q{q}.jpg"))[y1:y2, x1:x2]
        ax[i].set_title(f"JPEG q={q}")
    ax[i].imshow(cv2.cvtColor(part, cv2.COLOR_BGR2RGB), interpolation="nearest")
    ax[i].axis("off")
plt.savefig(os.path.join(FIG, "a2_jpeg_artifacts.png"), dpi=150, bbox_inches="tight")
plt.close()



# Крок 5. EXIF
print("\nКрок 5")



def read_exif(path):
    image = Image.open(path)
    exif = image.getexif()
    tags = {}
    for key, value in exif.items():
        tags[ExifTags.TAGS.get(key, key)] = value
    # витримка, ISO - в розділі Exif
    for key, value in exif.get_ifd(ExifTags.IFD.Exif).items():
        tags[ExifTags.TAGS.get(key, key)] = value
    return tags



rows = []
for name in ["photo.jpg", "texture.jpg", "object_light1.jpg", "object_light2.jpg"]:
    t = read_exif(os.path.join(DATA, name))
    exposure = f"1/{round(1 / float(t['ExposureTime']))} с"
    row = [name, t["Model"], f"{float(t['FocalLength'])} мм", f"{t['FocalLengthIn35mmFilm']} мм",
           f"f/{float(t['FNumber'])}", exposure, t["ISOSpeedRatings"], t["DateTimeOriginal"]]
    print(row)
    rows.append(row)

save_csv("a3_exif.csv", ["файл", "камера", "фокусна відстань", "екв. 35 мм", "діафрагма",
                         "витримка", "ISO", "дата"], rows)

# кадри набору (лекція 1: EXIF кожного кадру - до будь-якої обробки)
# у кадрів з відео EXIF немає - беру метадані відео, які зберіг extract_frames.py
video_info = {}
for name in os.listdir(os.path.join(DATA, "dataset")):
    if name.endswith("_frames.csv"):
```

```
        with open(os.path.join(DATA, "dataset", name), encoding="utf-8") as f:
            for row in csv.DictReader(f):
                video_info[row["файл"]] = row

rows = []
for name in sorted(os.listdir(os.path.join(DATA, "dataset"))):
    if not name.lower().endswith((".jpg", ".jpeg")):
        continue
    t = read_exif(os.path.join(DATA, "dataset", name))
    if "ExposureTime" in t:
        rows.append([name, t["Model"], f"{t['FocalLengthIn35mmFilm']} мм", f"1/{round(1 /
float(t['ExposureTime']))} с",
                     t["ISOSpeedRatings"], t["DateTimeOriginal"], "фото"])
    else:
        v = video_info[name]
        # час кадру = початок запису + час у відео
        moment = datetime.fromisoformat(v["початок запису"]) + timedelta(seconds=float(v["час у
відео, с"]))
        rows.append([name, v["камера"], "-", "-", "-", moment.strftime("%Y:%m:%d %H:%M:%S.%f")[:-4],
                     f"кадр {v['кадр']} з {v['відео']} (у відео немає витримки й ISO)"])
print("кадрів набору:", len(rows))
save_csv("a3_exif_dataset.csv", ["файл", "камера", "екв. 35 мм", "витримка", "ISO", "дата",
"джерело"], rows)
```

A.3. Частина B – part_b.py

```
# ЛР1, частина B, кроки 8-11
# межі - перцентилі 1..99 пікселів об'єкта на кадрі 1, ті самі межі на кадрі 2

import os

import cv2
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from utils import DATA, FIG, accuracy, iou, save_csv

np.random.seed(42)

frame1 = cv2.imread(os.path.join(DATA, "object_light1.jpg"))   # вікно
frame2 = cv2.imread(os.path.join(DATA, "object_light2.jpg"))   # лампа
mask1 = cv2.imread(os.path.join(DATA, "object_mask.png"), cv2.IMREAD_GRAYSCALE)
mask2 = cv2.imread(os.path.join(DATA, "object_mask2.png"), cv2.IMREAD_GRAYSCALE)  # телефон зсунувся
print("об'єкт займає", round((mask1 > 0).mean() * 100, 1), "% кадру")



def find_bounds(image, mask, channels, q=1):
    # перцентилі q..100-q; інші канали 0..255
    low = np.array([0, 0, 0], dtype=np.uint8)
    high = np.array([255, 255, 255], dtype=np.uint8)
    for ch in channels:
        values = image[:, :, ch][mask > 0]
        low[ch] = int(np.percentile(values, q))
        high[ch] = int(np.percentile(values, 100 - q))
    return low, high



spaces = [
    ["RGB", None, [0, 1, 2]],
    ["HSV", cv2.COLOR_BGR2HSV, [0, 1, 2]],
    ["HSV (лише H, S)", cv2.COLOR_BGR2HSV, [0, 1]],
    ["Lab (лише a, b)", cv2.COLOR_BGR2Lab, [1, 2]],
]
# H в OpenCV 0..179, для червоного треба було б два інтервали

bounds_rows = []
metric_rows = []
results = {}

for name, code, channels in spaces:
    if code is None:
        conv1, conv2 = frame1, frame2
    else:
        conv1 = cv2.cvtColor(frame1, code)
        conv2 = cv2.cvtColor(frame2, code)
```

```
    # межі по кадру 1
    low, high = find_bounds(conv1, mask1, channels)
    bounds_rows.append([name, str(low.tolist()), str(high.tolist())])


    pred1 = cv2.inRange(conv1, low, high)
    pred2 = cv2.inRange(conv2, low, high)
    results[name] = (pred1, pred2)


    iou1, iou2 = iou(pred1, mask1), iou(pred2, mask2)
    acc1, acc2 = accuracy(pred1, mask1), accuracy(pred2, mask2)
    metric_rows.append([name, round(iou1, 3), round(acc1, 3), round(iou2, 3), round(acc2, 3)])
    print(f"{name:16s} межі {low.tolist()}..{high.tolist()}  кадр 1: IoU = {iou1:.3f}  кадр 2: IoU =
{iou2:.3f}")

# Крок 11. Таблиця
save_csv("b_thresholds.csv", ["простір", "нижні межі", "верхні межі"], bounds_rows)
save_csv("b_metrics.csv", ["простір", "IoU, кадр 1", "accuracy, кадр 1", "IoU, кадр 2", "accuracy,
кадр 2"],
         metric_rows)

# accuracy порожньої маски
print("accuracy порожньої маски на кадрі 2:", round(accuracy(np.zeros_like(mask2), mask2), 3))


# чи залежить висновок від вибору перцентилів
print("\nIoU на кадрі 2 при різних перцентилях")
sens_rows = []
for name, code, channels in spaces:
    if code is None:
        conv1, conv2 = frame1, frame2
    else:
        conv1 = cv2.cvtColor(frame1, code)
        conv2 = cv2.cvtColor(frame2, code)
    row = [name]
    for q in [1, 2, 5]:
        low, high = find_bounds(conv1, mask1, channels, q)
        row.append(float(round(iou(cv2.inRange(conv2, low, high), mask2), 3)))
    print(name, row[1:])
    sens_rows.append(row)
save_csv("b_sensitivity.csv", ["простір", "1-99", "2-98", "5-95"], sens_rows)



# рисунки
fig, ax = plt.subplots(1, 4, figsize=(16, 5))
pictures = [cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB), cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB), mask1,
mask2]
titles = ["кадр 1 (денне світло)", "кадр 2 (лампа)", "маска кадру 1", "маска кадру 2"]
for i in range(4):
    ax[i].imshow(pictures[i], cmap="gray")
    ax[i].set_title(titles[i])
    ax[i].axis("off")
plt.savefig(os.path.join(FIG, "b_inputs.png"), dpi=150, bbox_inches="tight")
plt.close()



def errors_picture(pred, gt):
    # зелений - ок, червоний - зайве, синій - пропущено
    pred = pred > 0
    gt = gt > 0
    pic = np.zeros((gt.shape[0], gt.shape[1], 3), dtype=np.uint8)
    pic[pred & gt] = [0, 170, 0]
    pic[pred & ~gt] = [230, 30, 30]
    pic[~pred & gt] = [40, 80, 255]
    return pic



fig, ax = plt.subplots(len(spaces), 2, figsize=(8, 3.2 * len(spaces)))
for i, (name, code, channels) in enumerate(spaces):
    pred1, pred2 = results[name]
    ax[i, 0].imshow(errors_picture(pred1, mask1))
    ax[i, 0].set_title(f"{name}, кадр 1: IoU = {iou(pred1, mask1):.2f}", fontsize=9)
    ax[i, 1].imshow(errors_picture(pred2, mask2))
    ax[i, 1].set_title(f"{name}, кадр 2: IoU = {iou(pred2, mask2):.2f}", fontsize=9)
    ax[i, 0].axis("off")
    ax[i, 1].axis("off")
fig.suptitle("зелений - правильно, червоний - зайве, синій - пропущено")
plt.tight_layout()
plt.savefig(os.path.join(FIG, "b_masks.png"), dpi=150, bbox_inches="tight")
plt.close()
```

```
names = [row[0] for row in metric_rows]
x = np.arange(len(names))
plt.figure(figsize=(9, 4.5))
plt.bar(x - 0.2, [row[1] for row in metric_rows], 0.4, label="кадр 1 (тут підбирались межі)")
plt.bar(x + 0.2, [row[3] for row in metric_rows], 0.4, label="кадр 2 (ті самі межі)")
plt.xticks(x, names)
plt.ylabel("IoU з еталонною маскою")
plt.ylim(0, 1)
plt.title("Стабільність порогів при зміні освітлення")
plt.grid(axis="y", alpha=0.3)
plt.legend()
plt.savefig(os.path.join(FIG, "b_iou.png"), dpi=150, bbox_inches="tight")
plt.close()
```

A.4. Частина C – part_c.py

```
# ЛР1, частина C, кроки 12-14


import os
import time


import cv2
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from skimage.metrics import structural_similarity


from utils import DATA, FIG, psnr, save_csv


np.random.seed(42)


photo = cv2.imread(os.path.join(DATA, "photo.jpg"))
texture = cv2.imread(os.path.join(DATA, "texture.jpg"))



# Крок 12. Аліасинг
print("Крок 12")



def downscale_4_ways(image):
    w, h = image.shape[1] // 4, image.shape[0] // 4
    nearest = cv2.resize(image, (w, h), interpolation=cv2.INTER_NEAREST)
    blurred = cv2.GaussianBlur(image, (0, 0), 2)
    blur_nearest = cv2.resize(blurred, (w, h), interpolation=cv2.INTER_NEAREST)
    area = cv2.resize(image, (w, h), interpolation=cv2.INTER_AREA)
    # піраміда: pyrDown = гаус 5x5 + кожен другий піксель, два рівні = у 4 рази
    pyramid = cv2.pyrDown(cv2.pyrDown(image))
    return nearest, blur_nearest, area, pyramid



# зонна пластинка, локальна частота f = r / N
N = 1024
yy, xx = np.mgrid[0:N, 0:N] - N / 2
r = np.sqrt(xx ** 2 + yy ** 2)
zone = (127.5 + 127.5 * np.cos(np.pi * r ** 2 / N)).astype(np.uint8)


# після зменшення в 4 рази Найквіст = 0.125; вище має бути сіре, std там = муар
r_small = r[::4, ::4]
zone_moire = (r_small / N > 0.15) & (r_small / N < 0.48)
zone_low = r_small / N < 0.075
low_std_original = zone[r < 0.075 * N].std()


rows = []
names = ["INTER_NEAREST", "GaussianBlur + NEAREST", "INTER_AREA", "pyrDown x2"]
for name, small in zip(names, downscale_4_ways(zone)):
    moire = small[zone_moire].std()
    kept = small[zone_low].std() / low_std_original * 100
    print(f"{name:24s} муар (std) = {moire:.1f}   збережений контраст = {kept:.1f} %")
    rows.append([name, round(moire, 1), round(kept, 1)])
save_csv("c1_aliasing.csv", ["метод", "std у зоні f > f_Найквіста", "збережений контраст низьких
частот, %"], rows)


# рисунок
fig, ax = plt.subplots(2, 6, figsize=(24, 9))
```

```
for row, (title, image) in enumerate([("зонна пластинка", zone), ("вишивка (texture.jpg)",
texture)]):
    if image.ndim == 3:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    h, w = image.shape[:2]
    # без antialiased муар видно вже на самому оригіналі
    ax[row, 0].imshow(image, cmap="gray", interpolation="antialiased")
    ax[row, 0].set_title(f"{title}\n{w}×{h}")
    for i, small in enumerate(downscale_4_ways(image)):
        ax[row, i + 1].imshow(small, cmap="gray", interpolation="nearest", extent=(0, w, h, 0))
        ax[row, i + 1].set_title(f"{names[i]}\n{small.shape[1]}×{small.shape[0]}")
    # шматок 1:1
    if row == 0:
        piece = image[448:576, 806:934]
    else:
        piece = image[936:1064, 686:814]
    ax[row, 5].imshow(piece, cmap="gray", interpolation="nearest")
    ax[row, 5].set_title("фрагмент оригіналу 128×128")
    for a in ax[row]:
        a.axis("off")
plt.savefig(os.path.join(FIG, "c1_aliasing.png"), dpi=150, bbox_inches="tight")
plt.close()



# Крок 13. Інтерполяція
print("\nКрок 13")
methods = [["INTER_NEAREST", cv2.INTER_NEAREST],
           ["INTER_LINEAR", cv2.INTER_LINEAR],
           ["INTER_CUBIC", cv2.INTER_CUBIC],
           ["INTER_LANCZOS4", cv2.INTER_LANCZOS4]]

# шматок з горизонтом (на самому лісі й на вишивці всі методи виходять однакові)
crop = photo[500:1012, 760:1272]
small = cv2.resize(crop, (64, 64), interpolation=cv2.INTER_AREA)

fig, ax = plt.subplots(2, 3, figsize=(13, 9))
ax = ax.ravel()
ax[0].imshow(cv2.cvtColor(crop, cv2.COLOR_BGR2RGB))
ax[0].set_title("оригінальний фрагмент 512×512")
ax[1].imshow(cv2.cvtColor(small, cv2.COLOR_BGR2RGB), interpolation="nearest")
ax[1].set_title("зменшений у 8 разів: 64×64")

# ще 5 шматків уздовж горизонту - для розкиду
x_list = [0, 380, 760, 1140, 1536]

rows = []
for i, (name, flag) in enumerate(methods):
    big = cv2.resize(small, (512, 512), interpolation=flag)
    p = psnr(crop, big)
    s = structural_similarity(crop, big, channel_axis=2)
    ax[i + 2].imshow(cv2.cvtColor(big, cv2.COLOR_BGR2RGB))
    ax[i + 2].set_title(f"{name}\nPSNR {p:.2f} дБ, SSIM {s:.3f}")

    psnr_list = []
    ssim_list = []
    for x0 in x_list:
        part = photo[500:1012, x0:x0 + 512]
        part_small = cv2.resize(part, (64, 64), interpolation=cv2.INTER_AREA)
        part_big = cv2.resize(part_small, (512, 512), interpolation=flag)
        psnr_list.append(psnr(part, part_big))
        ssim_list.append(structural_similarity(part, part_big, channel_axis=2))

    # час: 20 замірів
    times = []
    for _ in range(20):
        start = time.perf_counter()
        cv2.resize(small, (512, 512), interpolation=flag)
        times.append((time.perf_counter() - start) * 1000)

    print(f"{name:15s} PSNR = {np.mean(psnr_list):.2f} ± {np.std(psnr_list):.2f} дБ  "
          f"SSIM = {np.mean(ssim_list):.3f} ± {np.std(ssim_list):.3f}  "
          f"час = {np.median(times):.3f} мс (std {np.std(times):.3f})")
    rows.append([name, round(np.mean(psnr_list), 2), round(np.std(psnr_list), 2),
                 round(np.mean(ssim_list), 3), round(np.std(ssim_list), 3),
                 round(np.median(times), 3), round(np.std(times), 3)])
for a in ax:
    a.axis("off")
plt.savefig(os.path.join(FIG, "c2_interpolation.png"), dpi=150, bbox_inches="tight")
plt.close()
```

```
save_csv("c2_interpolation.csv", ["метод", "PSNR середнє, дБ", "PSNR std", "SSIM середнє", "SSIM
std",
                                  "час медіана, мс", "час std"], rows)



# Крок 14. Гамма
print("\nКрок 14")
gammas = [0.5, 1.0, 2.2]

# out = 255 * (in / 255) ^ g
x = np.arange(256)
plt.figure(figsize=(5.5, 5))
for g in gammas:
    plt.plot(x, 255 * (x / 255) ** g, label=f"γ = {g}")
plt.xlabel("вхідне значення, рівні 0-255")
plt.ylabel("вихідне значення, рівні 0-255")
plt.title("Передавальні характеристики")
plt.grid(alpha=0.3)
plt.legend()
plt.savefig(os.path.join(FIG, "c3_gamma_curves.png"), dpi=150, bbox_inches="tight")
plt.close()

fig, ax = plt.subplots(1, 3, figsize=(15, 4.5))
for i, g in enumerate(gammas):
    table = (255 * (np.arange(256) / 255) ** g).astype(np.uint8)
    corrected = cv2.LUT(photo, table)
    ax[i].imshow(cv2.cvtColor(corrected, cv2.COLOR_BGR2RGB))
    ax[i].set_title(f"γ = {g}")
    ax[i].axis("off")
plt.savefig(os.path.join(FIG, "c3_gamma_images.png"), dpi=150, bbox_inches="tight")
plt.close()

# освітленість L = (v / 255) ^ 2.2
rows = []
for v1, v2 in [[0, 255], [50, 200], [100, 150], [0, 100]]:
    mean_codes = (v1 + v2) / 2
    mean_light = ((v1 / 255) ** 2.2 + (v2 / 255) ** 2.2) / 2
    mean_light_code = 255 * mean_light ** (1 / 2.2)
    print(f"{v1} і {v2}: середнє кодів = {mean_codes}, середнє освітленостей =
{mean_light_code:.1f}")
    rows.append([v1, v2, mean_codes, round(mean_light_code, 1)])
save_csv("c3_gamma_average.csv", ["v1", "v2", "середнє кодів", "середнє освітленостей (у кодах)"],
rows)

# смуги 0/255 і сірі 128, 186 (дивитись у масштабі 100 %)
demo = np.zeros((200, 900), dtype=np.uint8)
demo[:, 0:300:2] = 255
demo[:, 300:600] = 128
demo[:, 600:900] = 186
cv2.imwrite(os.path.join(FIG, "c3_gamma_stripes.png"), demo)



# Сірий світ
print("\nСірий світ")
lamp = cv2.imread(os.path.join(DATA, "object_light2.jpg"))
means = lamp.reshape(-1, 3).mean(axis=0)
gain = means.mean() / means
balanced = np.clip(lamp * gain, 0, 255).astype(np.uint8)
means_after = balanced.reshape(-1, 3).mean(axis=0)
print("до:", means.round(1), "R/B =", round(means[2] / means[0], 2))
print("після:", means_after.round(1), "R/B =", round(means_after[2] / means_after[0], 2))
save_csv("c3_gray_world.csv", ["", "середнє B", "середнє G", "середнє R", "R/B"], [
    ["до"] + list(means.round(1)) + [round(means[2] / means[0], 2)],
    ["після"] + list(means_after.round(1)) + [round(means_after[2] / means_after[0], 2)],
])

fig, ax = plt.subplots(1, 2, figsize=(10, 6))
ax[0].imshow(cv2.cvtColor(lamp, cv2.COLOR_BGR2RGB))
ax[0].set_title("кадр під лампою")
ax[1].imshow(cv2.cvtColor(balanced, cv2.COLOR_BGR2RGB))
ax[1].set_title("після «сірого світу»")
ax[0].axis("off")
ax[1].axis("off")
plt.savefig(os.path.join(FIG, "c3_gray_world.png"), dpi=150, bbox_inches="tight")
plt.close()
```

A.5. Еталонна маска – make_mask.py

```
# маска об'єкта через GrabCut
```

```
# python make_mask.py object_light1.jpg object_mask.png 360 380 800 1330   (x y w h)

import os
import sys

import cv2
import numpy as np

from utils import DATA, FIG

image_name, mask_name = sys.argv[1], sys.argv[2]
x, y, w, h = [int(v) for v in sys.argv[3:7]]

img = cv2.imread(os.path.join(DATA, image_name))
mask = np.zeros(img.shape[:2], dtype=np.uint8)
bgd_model = np.zeros((1, 65), dtype=np.float64)
fgd_model = np.zeros((1, 65), dtype=np.float64)
cv2.grabCut(img, mask, (x, y, w, h), bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_RECT)

# 0, 2 - фон; 1, 3 - об'єкт
result = np.where((mask == 1) | (mask == 3), 255, 0).astype(np.uint8)
cv2.imwrite(os.path.join(DATA, mask_name), result)
print("об'єкт займає", round((result > 0).mean() * 100, 1), "% кадру")

# превью
preview = img.copy()
preview[result == 0] = preview[result == 0] // 3
cv2.rectangle(preview, (x, y), (x + w, y + h), (0, 255, 255), 3)
cv2.imwrite(os.path.join(FIG, mask_name.replace(".png", "_preview.jpg")), preview)
```

A.6. Кадри з відео – extract_frames.py

```
# кадри з відео для набору: n штук рівномірно, з сусідніх беру найрізкіший
# python extract_frames.py <відео> 10 rot

import csv
import json
import os
import subprocess
import sys

import cv2
import numpy as np

from utils import DATA

video_path = sys.argv[1]
n = int(sys.argv[2])
prefix = sys.argv[3]

out_dir = os.path.join(DATA, "dataset")
os.makedirs(out_dir, exist_ok=True)

video = cv2.VideoCapture(video_path)
total = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
fps = video.get(cv2.CAP_PROP_FPS)

# різкість = дисперсія лапласіана
frames = []
sharpness = []
while True:
    ok, frame = video.read()
    if not ok:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    frames.append(frame)
    sharpness.append(cv2.Laplacian(gray, cv2.CV_64F).var())
video.release()

# метадані відео (камера, час початку запису) - через ffprobe
info = subprocess.run(["ffprobe", "-v", "error", "-show_entries", "format_tags", "-of", "json",
video_path],
                      capture_output=True, text=True)
tags = json.loads(info.stdout)["format"]["tags"]
model = tags.get("com.apple.quicktime.model", "")
record_start = tags.get("com.apple.quicktime.creationdate", "")
print("камера:", model, " початок запису:", record_start)
```

```
rows = []
step = len(frames) / n
for k in range(n):
    center = int(step * (k + 0.5))
    # +-10 кадрів
    start = max(0, center - 10)
    end = min(len(frames), center + 11)
    best = start + int(np.argmax(sharpness[start:end]))
    name = f"{prefix}_{k + 1:02d}.jpg"
    cv2.imwrite(os.path.join(out_dir, name), frames[best], [cv2.IMWRITE_JPEG_QUALITY, 95])
    print(f"{name}: кадр {best} ({best / fps:.1f} с), різкість {sharpness[best]:.0f}")
    rows.append([name, os.path.basename(video_path), best, round(best / fps, 2), model,
record_start])

with open(os.path.join(out_dir, f"{prefix}_frames.csv"), "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["файл", "відео", "кадр", "час у відео, с", "камера", "початок запису"])
    writer.writerows(rows)
```